Creating a z/OS Data Set

Using TSO commands, ISPF panels, and Ansible tasks to create z/OS Data Sets

TEKsystems Global Services

WORKFORCE DEVELOPMENT

DATA SET ALLOCATION AND CATALOGUING

The MVS "room" of the house doesn't use filesystems, inodes, or any other familiar way of managing files. IBM developed their own approach to persisting data before UNIX was invented.

We'll cover that in more detail later, as it is pretty important to know. For now, we'll cover enough to get started using Ansible with z/OS.

Creating a data set (or file) is called *allocating* the data set.

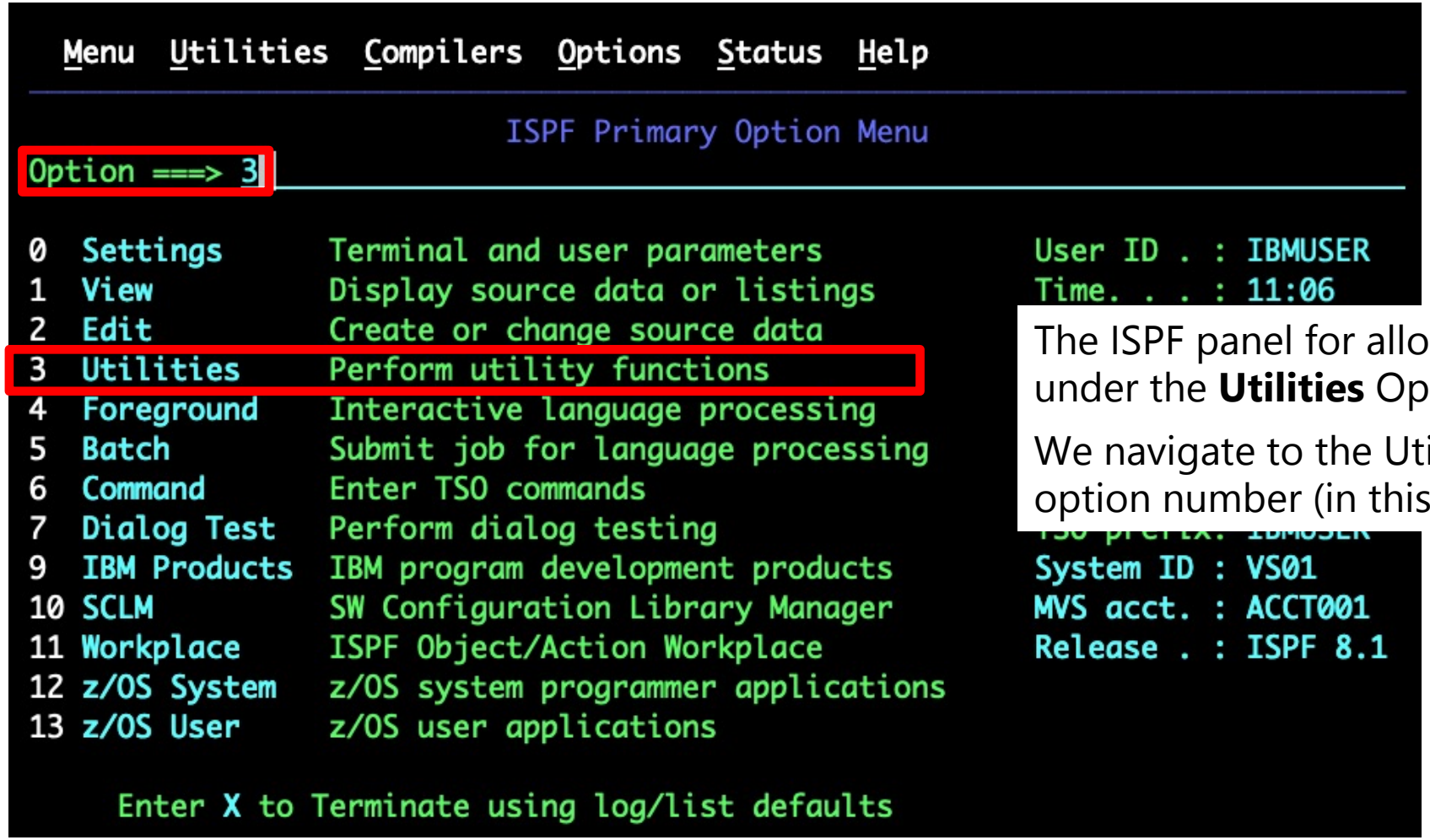
Adding information about the data set to the system catalog is called *cataloguing* the data set.

We can perform these operations interactively through ISPF (or plain TSO) or by writing JCL to run batch jobs that perform them.

With Ansible, we use the zos_data_set, zos_tso_command, and zos_job_submit modules from the ibm_zos_core collection to invoke these z/OS operations.

© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET



The ISPF panel for allocating data sets is located under the **Utilities** Option.

We navigate to the Utilities panel by entering its option number (in this case, 3) and pressing Enter.

© 2023 by Innovation In Software Corporation

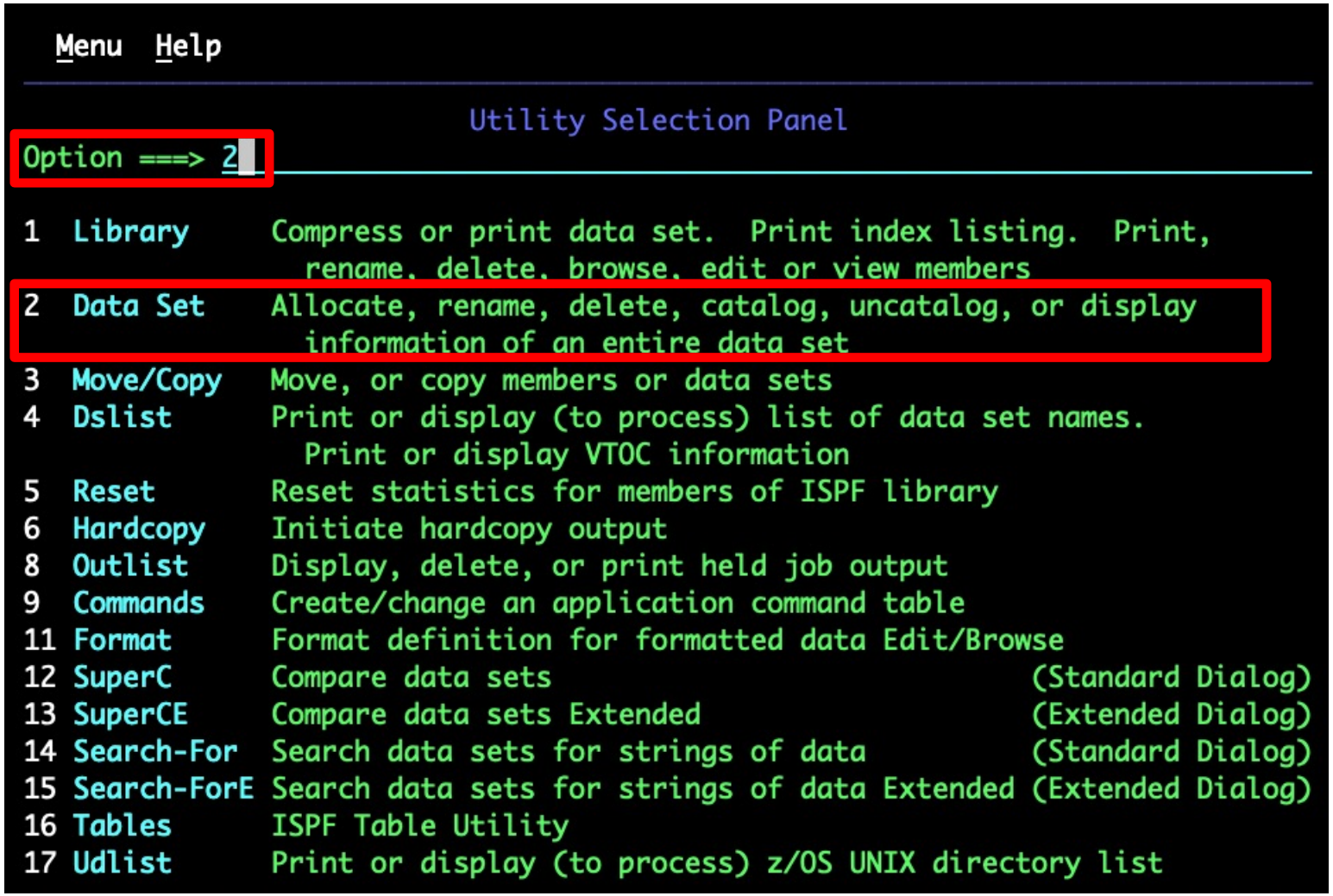
USING ISPF TO CREATE A DATA SET

Option 3 from the Primary Option Menu takes us to the Utility Selection Panel.

There are options to manage libraries (PDS and PDSE data sets), basic z/OS data set types (QSAM, etc.), and VSAM data sets. There are also utilities to compare data sets (SuperC and SuperCE) and to search for data sets containing particular values (Search-For and Search-ForE).

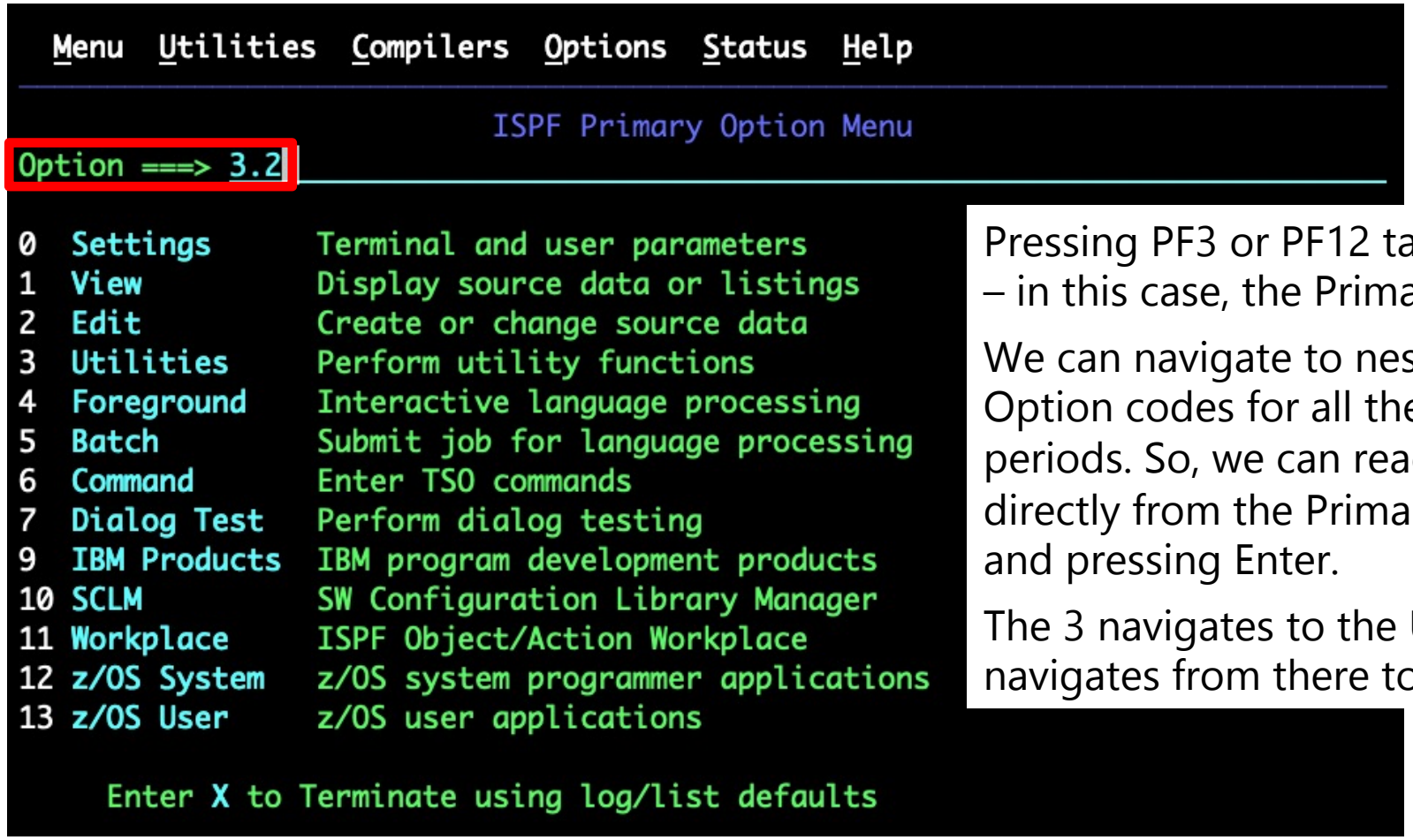
The Tables option pertains to customizing the ISPF Editor. It's out of scope for this class. The Udlist option pertains to Unix System Services (USS), also out of scope here.

You can probably surmise from the name of the option and its description that we're looking for **Option 2 – Data Set utilities.**



© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET



Pressing PF3 or PF12 takes us back where we came from – in this case, the Primary Option Menu.

We can navigate to nested ISPF panels by entering the Option codes for all the nested levels separated by periods. So, we can reach the Data Set Utilities panel directly from the Primary Option Menu by typing 3.2 and pressing Enter.

The 3 navigates to the Utility Selection Panel and the 2 navigates from there to the Data Set Utilities panel.

© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET

Here's the upper portion of the Data Set Utility panel. Functions available here include:

A – Allocate (create) data set

C – Catalog data set

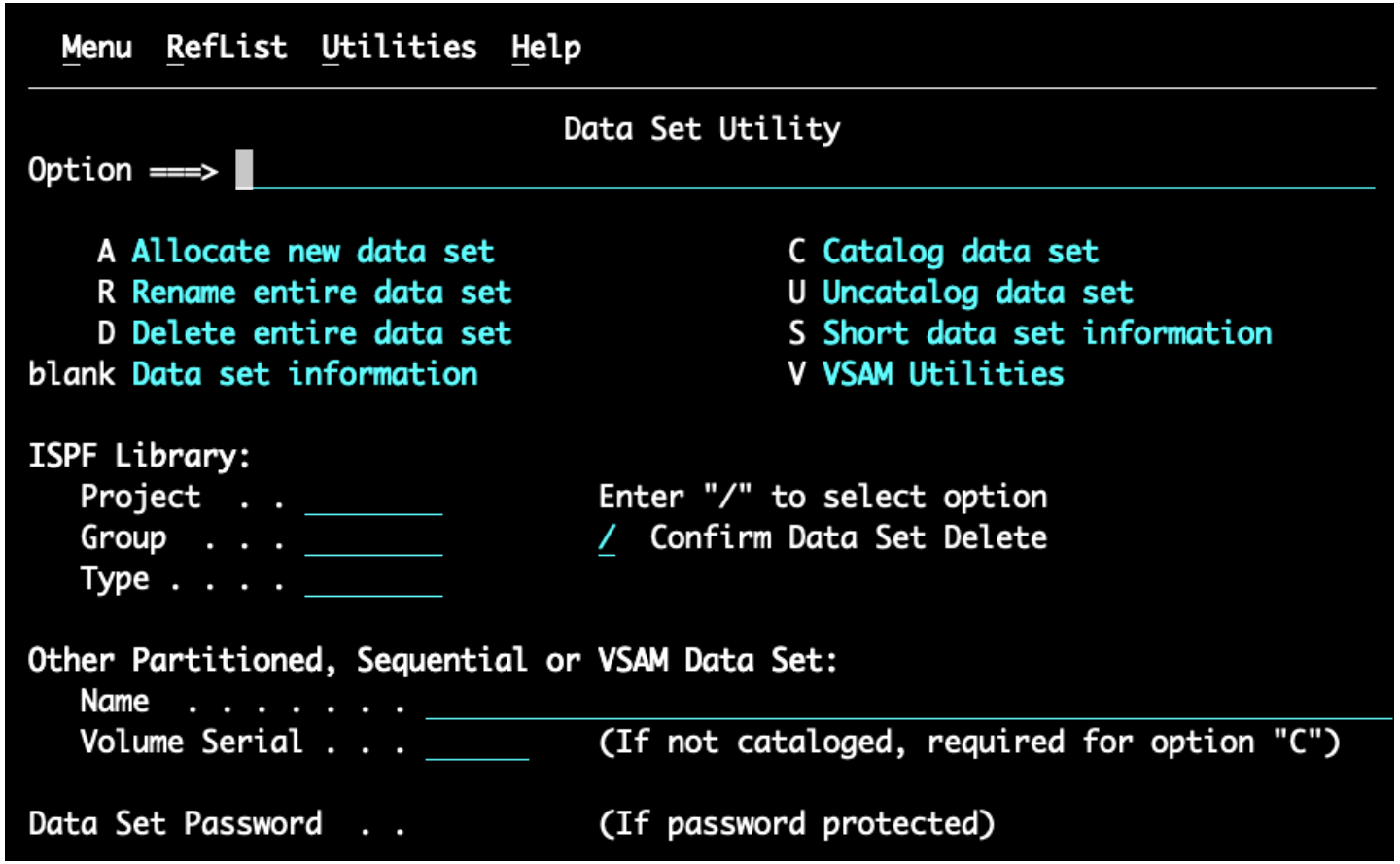
U – Uncatalog data set

D – Delete (uncatalog and unallocate)

blank, S – View detailed or summary information about a data set

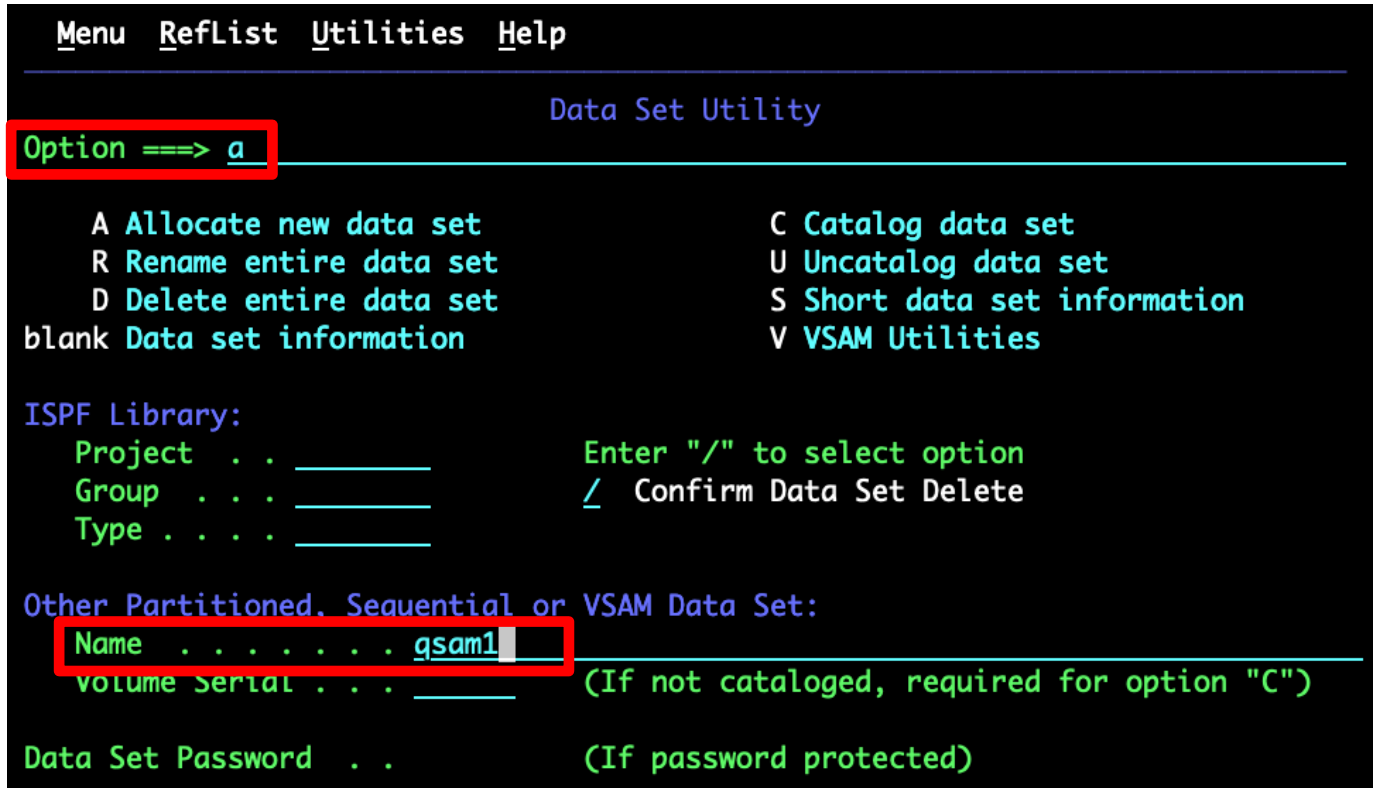
V – Navigate to the VSAM Utilities panel

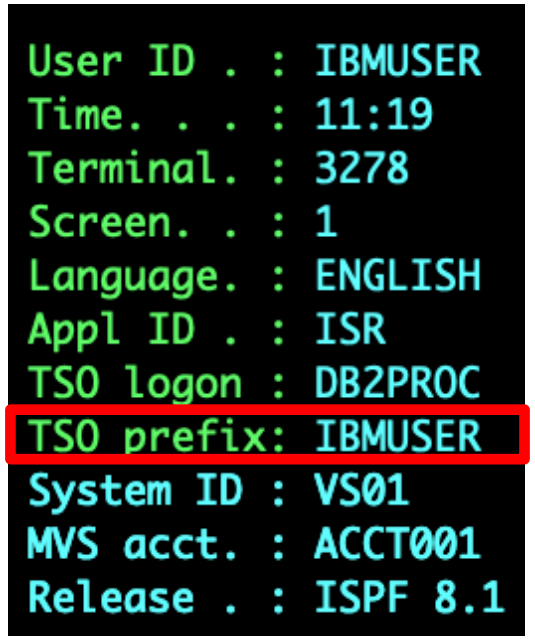
When working with z/OS applications, we use these functions frequently.



© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET



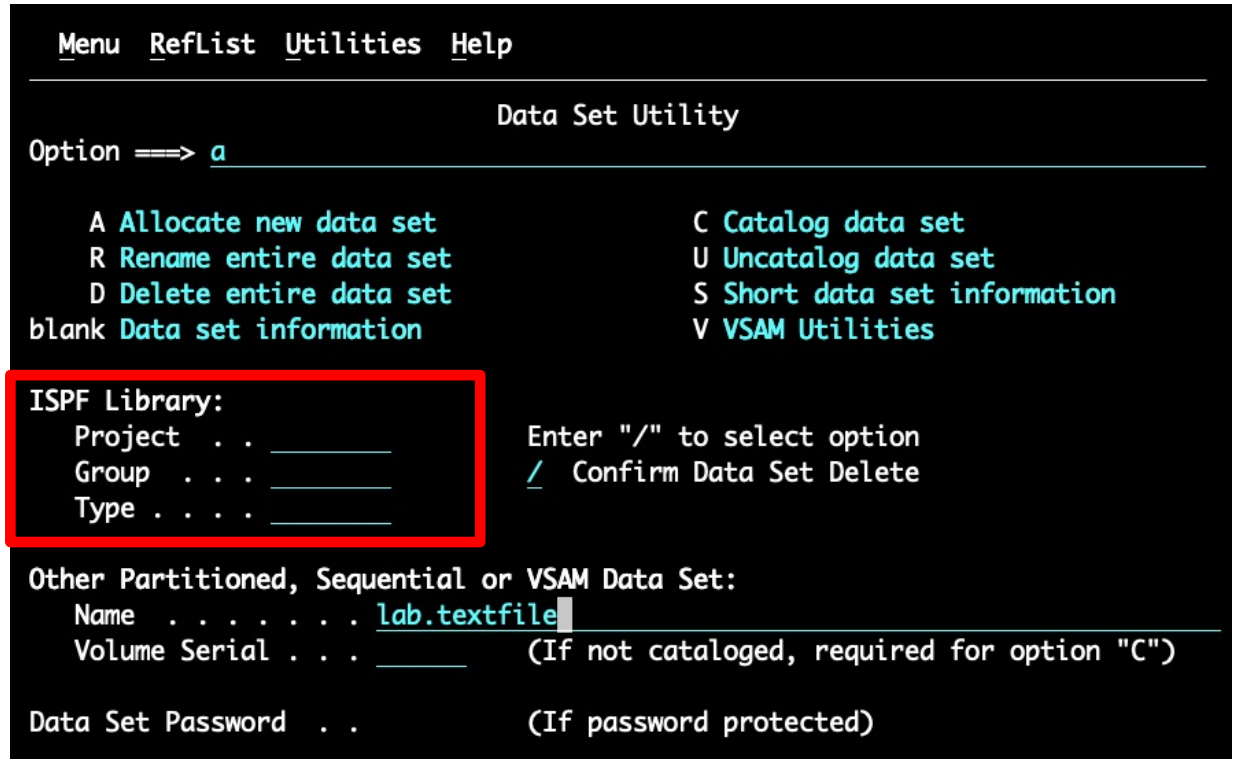


To identify the data set we want to create, we entered **qsam1** without enclosing the value in apostrophes (single quotes). TSO will change the text to uppercase and prepend the **TSO Prefix** as the HLQ, resulting in data set name **`IBMUSER.QSAM1`**. We could enter the full data set name within apostrophes: **`'IBMUSER.QSAM1'`**.

If we were authorized to create data sets under other HLQs, we could enter the full data set name enclosed within apostrophes, like **`'PROD45.SALES.COMM200'`**.

© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET



Project → High-Level Qualifier

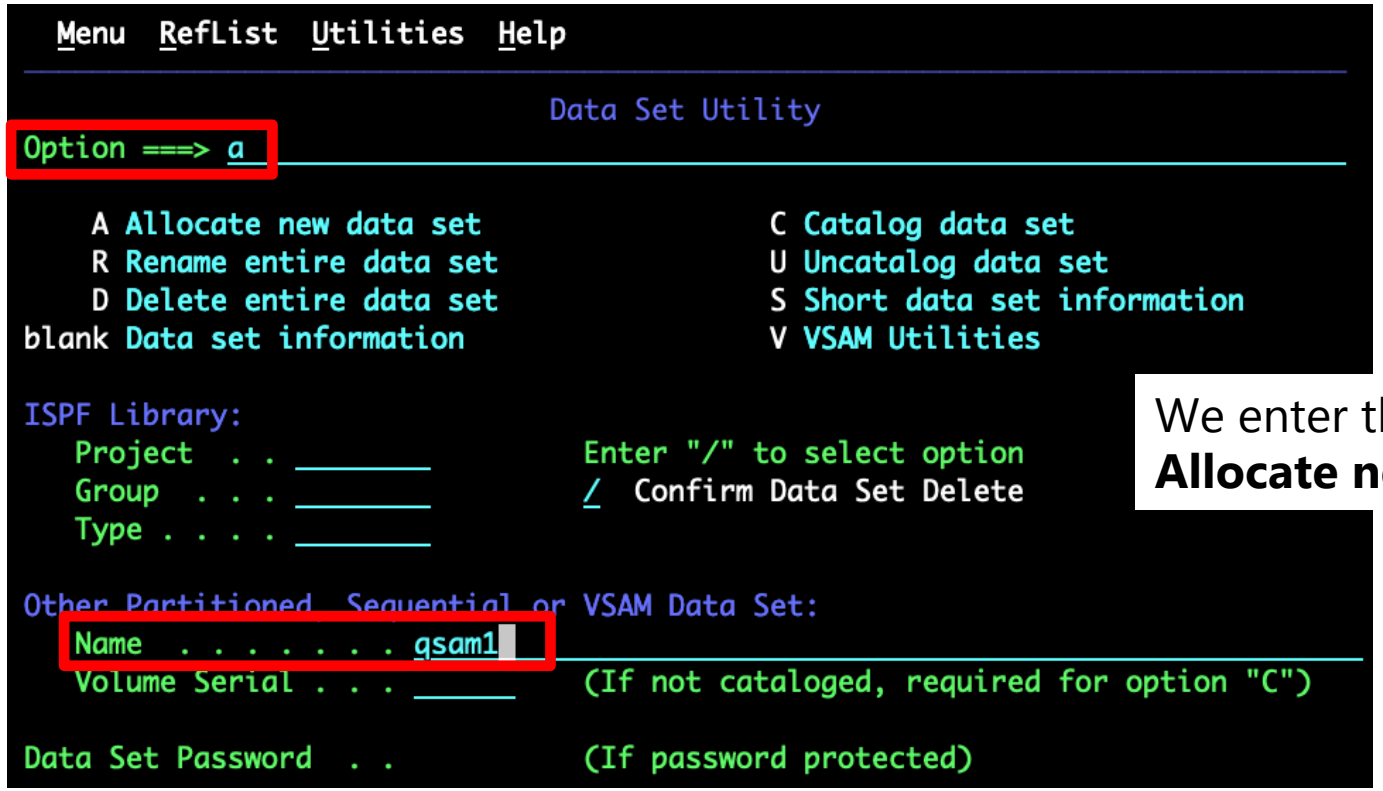
Group → Mid-Level Qualifier

Type → Low-Level Qualifier (Name)

A final comment on data set names and ISPF panel fields before we go on. Notice the section of the panel in the middle labeled **ISPF Library**. The **Project** field refers to the High-Level Qualifier. In this example, Project is **MATEVA**, the TSO Prefix. The **Group** field refers to a Mid-Level Qualifier. In this example, the value is **LAB**. The most common convention in z/OS installations is to have one Mid-Level qualifier, although it is valid to have more than one. Providing a single field for Mid-Level Qualifier reflects this convention. The **Type** field is the file name. In this example, the value is **TEXTFILE**. ISPF panels use this terminology consistently. We could enter those values here instead of in the **Other…Data Set** field. ISPF will convert data set name values to uppercase.

© 2023 by Innovation In Software Corporation

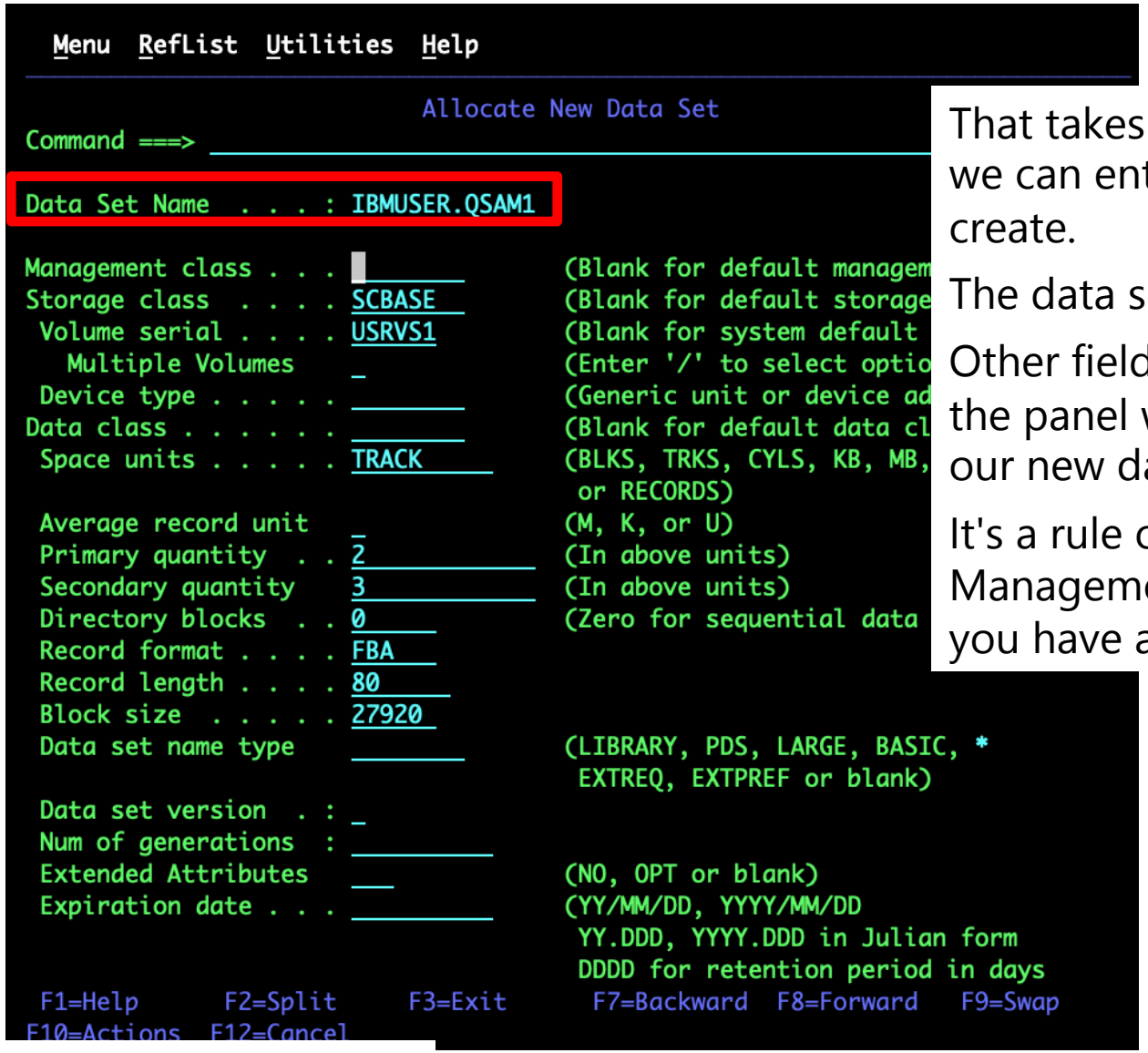
USING ISPF TO CREATE A DATA SET



We enter the option we want – in this case **A** for **Allocate new data set**, and press Enter.

© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET



That takes us to the Allocate New Data Set panel, where we can enter the attributes of the data set we want to create.

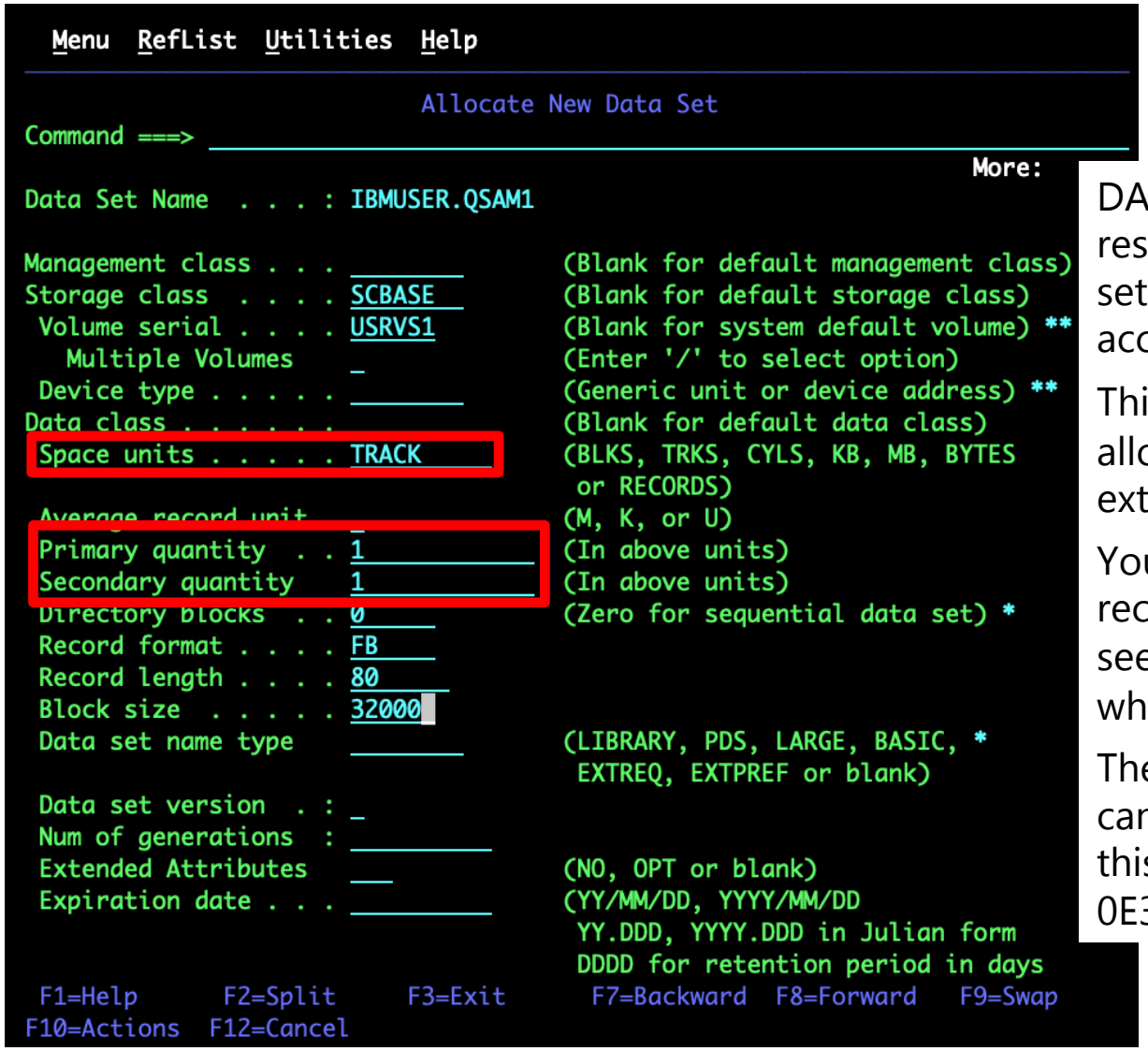
The data set name is pre-filled from the previous panel.

Other fields are pre-filled with values from the last time the panel was used. We want to override these values for our new data set.

It's a rule of thumb to take installation defaults for Management class, Storage class, and Data class unless you have a specific reason to use different values.

© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET



DASD space is allocated in *extents*. The primary extent is reserved for the data set when it is allocated. As the data set grows, secondary extents are created to accommodate the additional data.

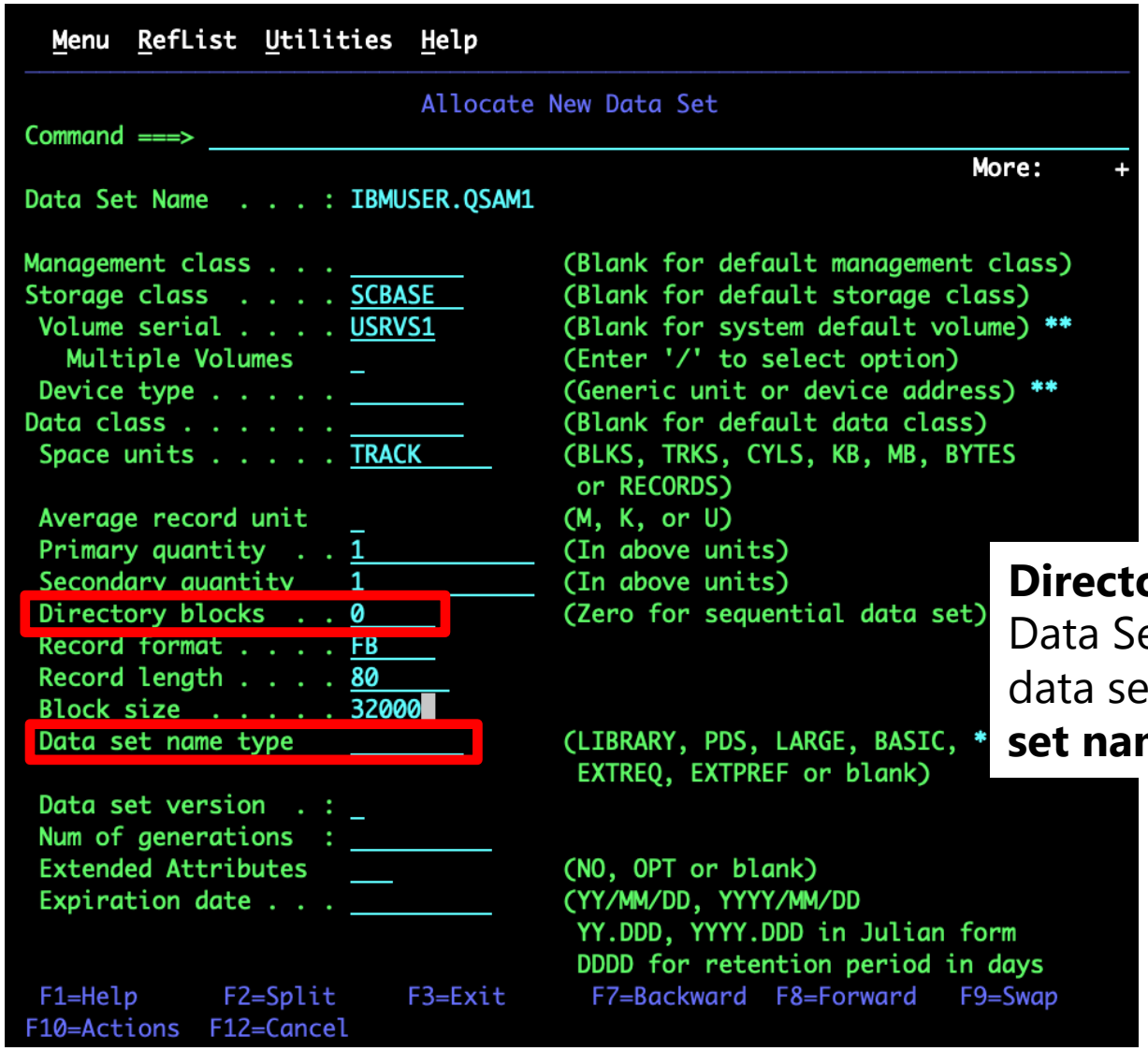
This example specifies space in terms of tracks, and allocates one track for the primary extent. Secondary extents will consist of 1 track each.

You can also specify the size in terms of cylinders, blocks, records, bytes, kilobytes, or megabytes. The brain in a jar sees everything in terms of IBM 3390 DASD devices, which have tracks and cylinders.

There is an installation-defined limit to how many extents can be added. When an application attempts to exceed this limit, it fails with an abend code of 0B37, 0D37 or 0E37 (these abend codes are equivalent).

© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET



Directory blocks pertain to libraries – that is, Partitioned Data Sets (PDS, PDSE types). For a "regular" sequential data set, specify zero directory blocks and leave the **Data set name type** field blank.

© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET

```
 Menu  RefList  Utilities  Help
                         Allocate New Data Set
Command ===>                                                  More:     +
Data Set Name  . . . : IBMUSER.QSAM1

Management class . . .  _______       (Blank for default management class)
Storage class  . . . .  SCBASE        (Blank for default storage c
 Volume serial . . . .  USRVS1        (Blank for system default vo
   Multiple Volumes     _             (Enter '/' to select option)
 Device type . . . . .  _______       (Generic unit or device addr
Data class . . . . . .  _______       (Blank for default data clas
 Space units . . . . .  TRACK         (BLKS, TRKS, CYLS, KB, MB, B
                                       or RECORDS)
 Average record unit    _             (M, K, or U)
 Primary quantity  . .  1             (In above units)
 Secondary quantity     1             (In above units)
 Directory blocks       0             (Zero for sequential data se
 Record format . . . .  FB
 Record length . . . .  80
 Block size  . . . . .  32000
 Data set name type     _______       (LIBRARY, PDS, LARGE, BASIC,
                                       EXTREQ, EXTPREF or blank)
 Data set version  . :  _
 Num of generations  :  _________
 Extended Attributes    ___           (NO, OPT or blank)
 Expiration date . . .  _________     (YY/MM/DD, YYYY/MM/DD
                                       YY.DDD, YYYY.DDD in Julian form
                                       DDDD for retention period in days
  F1=Help      F2=Split     F3=Exit      F7=Backward  F8=Forward   F9=Swap
F10=Actions  F12=Cancel
```

We're creating a data set with fixed-length records 80 bytes long, grouped into blocks, hence "record format" is "FB".

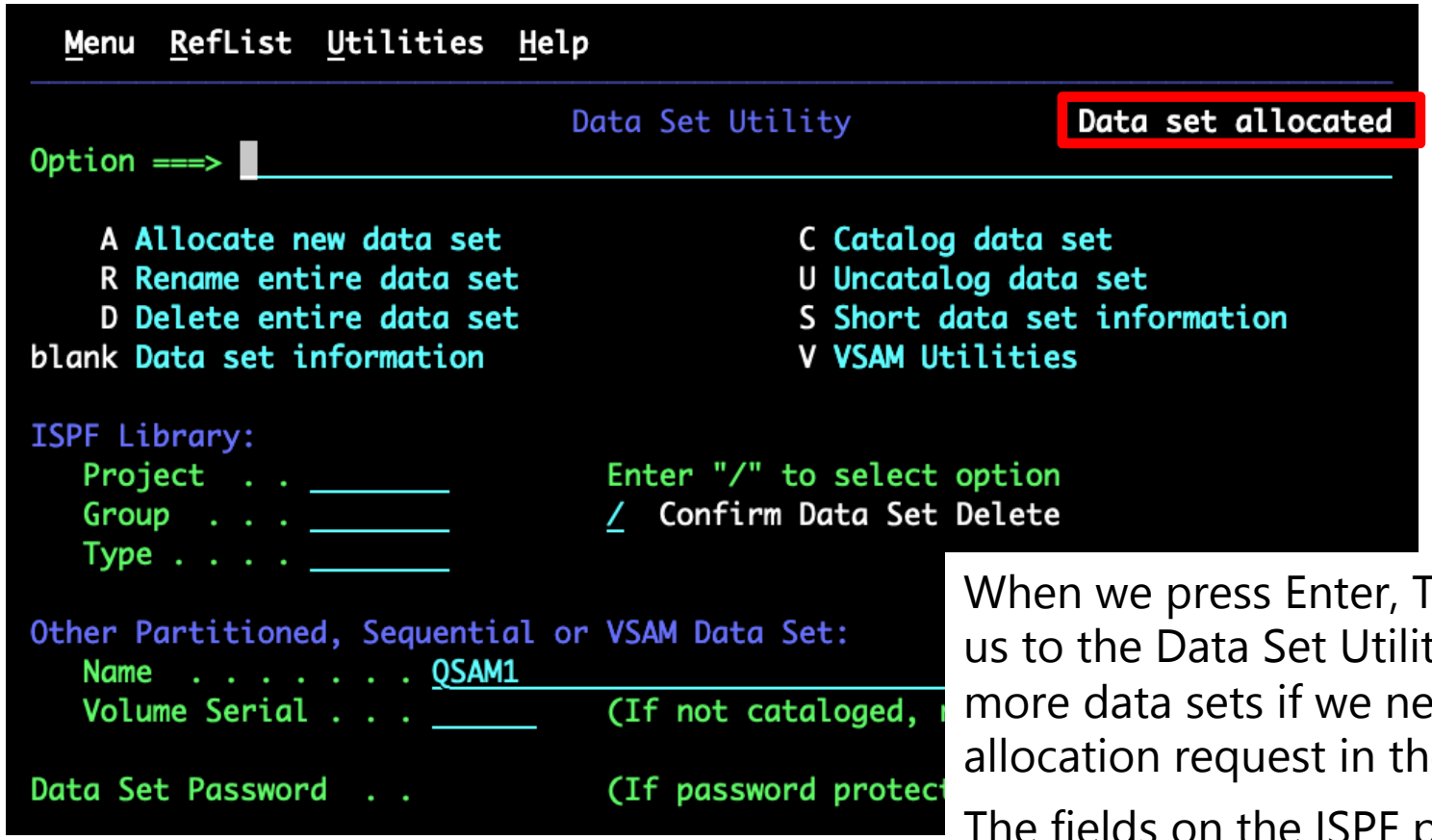
QSAM reads/writes **blocks** of data, also called *physical records*, and presents the data to application programs one *logical record* at a time. We specify 80 bytes as the logical record length and a block size that is an even multiple of the logical record length.

All this is virtualized today, but when DASD were physical devices the difference in I/O delay could be significant based on these settings.

The remaining fields pertain to other types of data sets or to additional attributes that may be associated with a data set.

© 2023 by Innovation In Software Corporation

USING ISPF TO CREATE A DATA SET



When we press Enter, TSO allocates the data set and returns us to the Data Set Utility panel, where we can work with more data sets if we need to. It displays the result of the allocation request in the upper right-hand corner.

The fields on the ISPF panels correspond to the same attributes we define as parameters on DD statements in JCL or as vars in an Ansible task.

©2023 by Innovation In Software Corporation

LAB 5: USING ISPF TO CREATE A QSAM DATA SET

Follow the instructions for Lab 5 in labs/labs.md in the course repo.

© 2023 by Innovation In Software Corporation

USING IBM_ZOS_CORE.ZOS_DATA_SET

Now let's see how to do the same thing using the ibm_zos_core collection in Ansible.

```
- name: Conditionally delete and then create a sequential data set
  hosts: zos
  gather_facts: false
  collections:
    - ibm.ibm_zos_core
  environment: "{{ environment_vars }}"
```

We start with a basic setup, telling Ansible that we want to use ibm_zos_core and bringing in the environment.

```
  tasks:
    - name: Set facts used in this playbook
      set_fact:
        data_set_name: IBMUSER.QSAM1
```

Now we put the data set name in a var

```
    - name: Delete data set if it exists
      zos_data_set:
        name: "{{ data_set_name }}"        # DELET
        type: seq                          # Match
        state: absent                      # DELET
        force: true
      register: result

    - name: Display result of data set deletion
      debug:
        msg: "{{ result }}"
```

If the data set already exists, z/OS won't let us create it. (Actually we can override that behavior, but for purposes of example let's handle it manually.)

So, we'll delete the data set if it exists.

Module zos_data_set will do this because of the "state: absent" specification.

© 2023 by Innovation In Software Corporation

USING IBM_ZOS_CORE.ZOS_DATA_SET

```
- name: Create sequential data set
  zos_data_set:
    name: "{{ data_set_name }}"        # CREATE: DD DSN=(...)
    type: seq                          # CREATE: DD DCB=(DSORG=PS)
    format: fb                         # CREATE: DD DCB=(RECFM=FB)
    record_length: 80                  # CREATE: DD DCB=(LRECL=80)
    block_size: 8000                   # CREATE: DD DCB=(BLKSIZE=32000)
    state: present                     # CREATE: DD DISP=(MOD,...)
    replace: true                      # CREATE: DD DISP(...,CATLG,...)
    space_type: trk                    # CREATE: DD SPACE=(TRK,...)
    space_primary: 1                   # CREATE: DD SPACE=(...,(1,...))
    space_secondary: 1                 # CREATE: DD SPACE=(...,(...,1))
  register: result

- name: Display result of data set creation
  debug:
    msg: "{{ result }}"
```

Now we can attempt to create the data set.

The comments to the right of the task show the corresponding values in z/OS Job Control Language (JCL). You've seen the equivalent settings on the ISPF Allocate Data Set panel.

You might notice we're setting the block size to 8000 rather than 32000. That's so we'll be able to tell that our Ansible playbook worked when we look at the data set information in ISPF.

© 2023 by Innovation In Software Corporation

USING IBM_ZOS_CORE.ZOS_DATA_SET

Here's the command to run the playbook

```
[ansible@ip-10-0-1-93 mainframe]$ ansible-playbook -i inventory.yml /home/ansible/mainframe/create_qsam.yml
```

```
TASK [Display result of data set deletion] *****
*******************************
ok: [mainframe] => {
    "msg": {
        "changed": true,
        "failed": false,
        "message": "",
        "names": [
            "IBMUSER.QSAM1"
        ]
    }
}

TASK [Create sequential data set] **************
*******************************
changed: [mainframe]

TASK [Display result of data set creation] *****
*******************************
ok: [mainframe] => {
    "msg": {
        "changed": true,
        "failed": false,
        "message": "",
        "names": [
            "IBMUSER.QSAM1"
        ]
    }
}

PLAY RECAP *************************************
*******************************
mainframe                  : ok=5    changed=2
```

This is what we see on the control node after the playbook has run.

"failed: false" means no errors occurred on the Ansible side.

"changed: true" means the task was successful, as far as Ansible can tell.

© 2026 by Innovation In Software Corporation

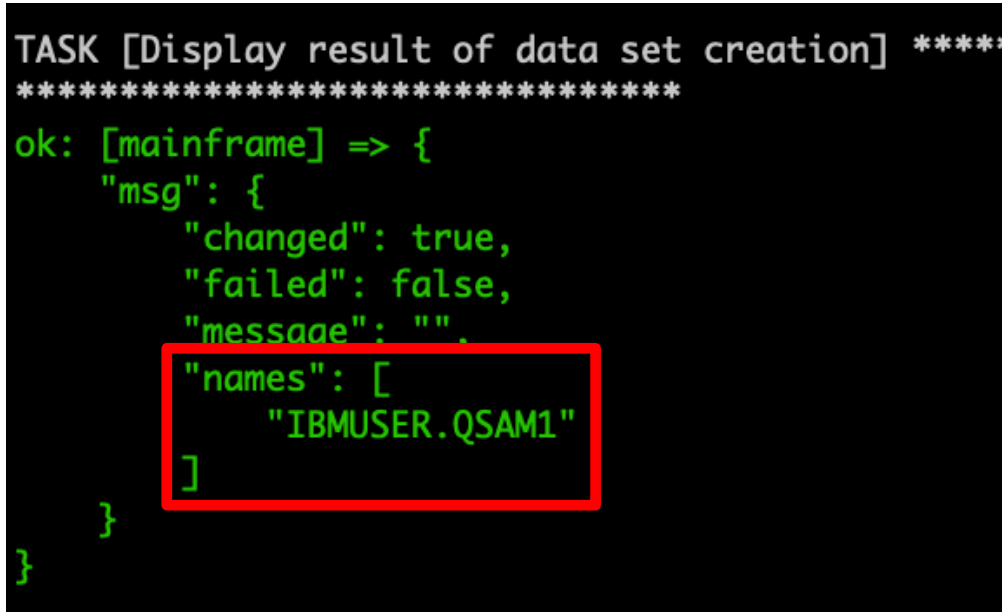
USING IBM_ZOS_CORE.ZOS_DATA_SET

```
                          Data Set Information
Command ===>

Data Set Name . . . . : IBMUSER.QSAM1

General Data                          Current Allocation
 Management class . . : **None**       Allocated tracks  . : 1
 Storage class  . . . : SCBASE         Allocated extents . : 1
  Volume serial . . . : USRVS1
  Device type . . . . : 3390
 Data class . . . . . : **None**
  Organization  . . . : PS             Current Utilizatior
  Record format . . . : FB              Used tracks . . .
  Record length . . . : 80              Used extents  . .
  Block size  . . . . : 8000
  1st extent tracks . : 1
  Secondary tracks  . : 1             Dates
  Data set name type  :                Creation date . . . : 2026/01/25
  Data set encryption : NO             Referenced date . . : ***None***
                                       Expiration date . . : ***None***

  SMS Compressible  . : NO
```

When we return to ISPF and check the attributes of the data set, we see that our block size value of 8000 was applied. We're sure the Ansible playbook worked.

© 2023 by Innovation In Software Corporation

USING IBM_ZOS_CORE.ZOS_DATA_SET



Notice that in the results from running the playbook, create_qsam_multiple, the "names" item is a list.

This suggests we can operate on multiple data sets in a single task.

Let's explore that.

© 2023 by Innovation In Software Corporation

USING IBM_ZOS_CORE.ZOS_DATA_SET

```
---
# This playbook creates multiple QSAM data sets (sequential files)
# on the z/OS instance.

- name: Conditionally delete and create sequential data sets
  hosts: zos
  gather_facts: false
  collections:
    - ibm.ibm_zos_core
  environment: "{{ environment_vars }}"

  tasks:
    - name: Set facts used in this playbook
      set_fact:
        zos_username: IBMUSER

    - name: Create sequential data sets
      zos_data_set:
        name: "{{ item }}"          # CREATE: DD DSN=(...)
        type: seq                   # CREATE: DD DCB=(DSORG=PS)
        format: fb                  # CREATE: DD DCB=(RECFM=FB)
        record_length: 80           # CREATE: DD DCB=(LRECL=80)
        block_size: 8000            # CREATE: DD DCB=(BLKSIZE=32000)
        state: present              # CREATE: DD DISP=(MOD,...)
        replace: true               # CREATE: DD DISP(...,CATLG,...)
        space_type: trk             # CREATE: DD SPACE=(TRK,...)
        space_primary: 1            # CREATE: DD SPACE=(...,(1,...))
        space_secondary: 1          # CREATE: DD SPACE=(...,(...,1))
      loop:
        - "{{ zos_username }}.QSAM1"
        - "{{ zos_username }}.QSAM2"
        - "{{ zos_username }}.QSAM3"
      register: result

    - name: Display result of data set creation
      debug:
        msg: "{{ result }}"
```

Here we've modified the create-qsam playbook to create three QSAM data sets with the same attributes.

We do this using the "loop" feature of Ansible.

We also removed the explicit "delete" task because we can specify "replace: true" on the "create" task.

© 2023 by Innovation In Software Corporation

USING IBM_ZOS_CORE.ZOS_DATA_SET

```
TASK [Display result of data set creation] *************
ok: [mainframe] => {
    "msg": {
        "changed": true,
        "msg": "All items completed",
        "results": [
            {
                "ansible_loop_var": "item",
                "changed": true,
                "failed": false,
                "invocation": {
                    "module_args": {
                        "batch": null,
                        "block_size": 8000,
                        "directory_blocks": null,
                        "force": false,
                        "format": "fb",
                        "key_length": null,
                        "key_offset": null,
                        "name": "IBMUSER.QSAM1",
                        "record_format": "fb",
                        "record_length": 80,
                        "replace": true,
                        "sms_data_class": null,
                        "sms_management_class": null,
                        "sms_storage_class": null,
                        "space_primary": 1,
                        "space_secondary": 1,
                        "space_type": "trk",
                        "state": "present",
                        "tmp_hlq": null,
                        "type": "seq",
                        "volumes": null
                    }
                },
                "item": "IBMUSER.QSAM1",
                "message": "",
                "names": [
                    "IBMUSER.QSAM1"
                ]
```

The result of running playbook create-qsam-multiple shows the attributes of the data sets that were created. All three data sets have the same attributes.

```
                },
                "item": "IBMUSER.QSAM2",
                "message": "",
                "names": [
                    "IBMUSER.QSAM2"
                ]
```

```
                "item": "IBMUSER.QSAM3",
                "message": "",
                "names": [
                    "IBMUSER.QSAM3"
                ]
```

© 2023 by Innovation In Software Corporation

LAB 6: USING ANSIBLE TO CREATE A QSAM DATA SET

Follow the instructions for Lab 6 in labs/labs.md in the course repo.

© 2023 by Innovation In Software Corporation

USING IBM_ZOS_CORE.ZOS_DATA_SET

What if we want to create data sets that have different attributes?

There are many use cases for this when we're provisioning environments for particular uses.

We might need to set up an environment for an application developer who is joining the organization. Let's say her team works with COBOL and REXX, and their application uses DB2.

In that case each developer on the team would need a source library for COBOL, one for REXX, a program object library, and a library for QMF scripts.

These libraries are a different type of z/OS data set known as a PDSE, or Partitioned Data Set Extended.

We'll defer creating these types of datasets until we've learned a little more about how they are implemented and used.

© 2023 by Innovation In Software Corporation

LAB 7: USING JCL TO CREATE A QSAM DATA SET

Follow the instructions for Lab 7 in labs/labs.md in the course repo.

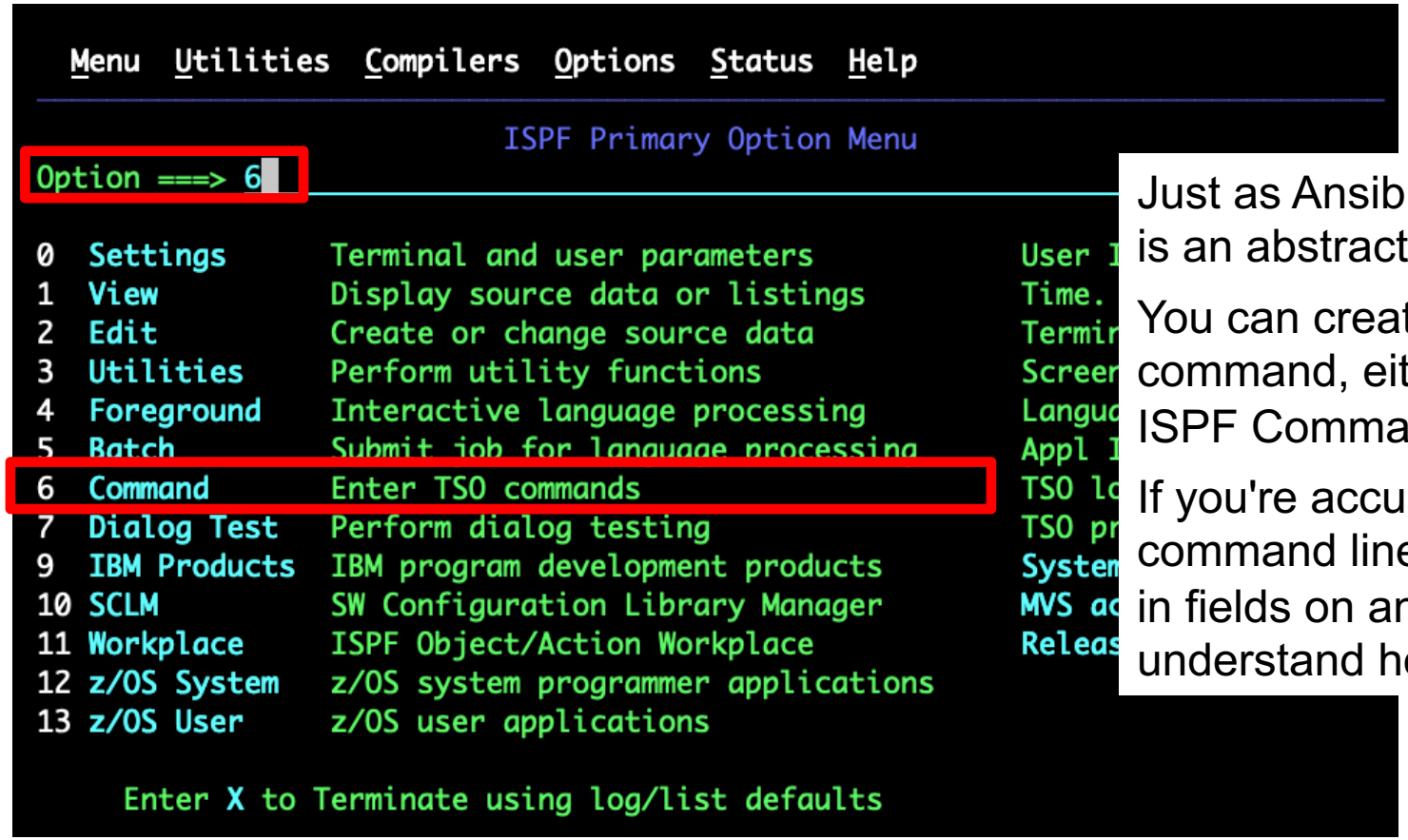
© 2023 by Innovation In Software Corporation

LAB 8: SUBMIT JCL FROM ANSIBLE

Follow the instructions for Lab 8 in labs/labs.md in the course repo.

© 2023 by Innovation In Software Corporation

USING TSO TO CREATE A DATA SET



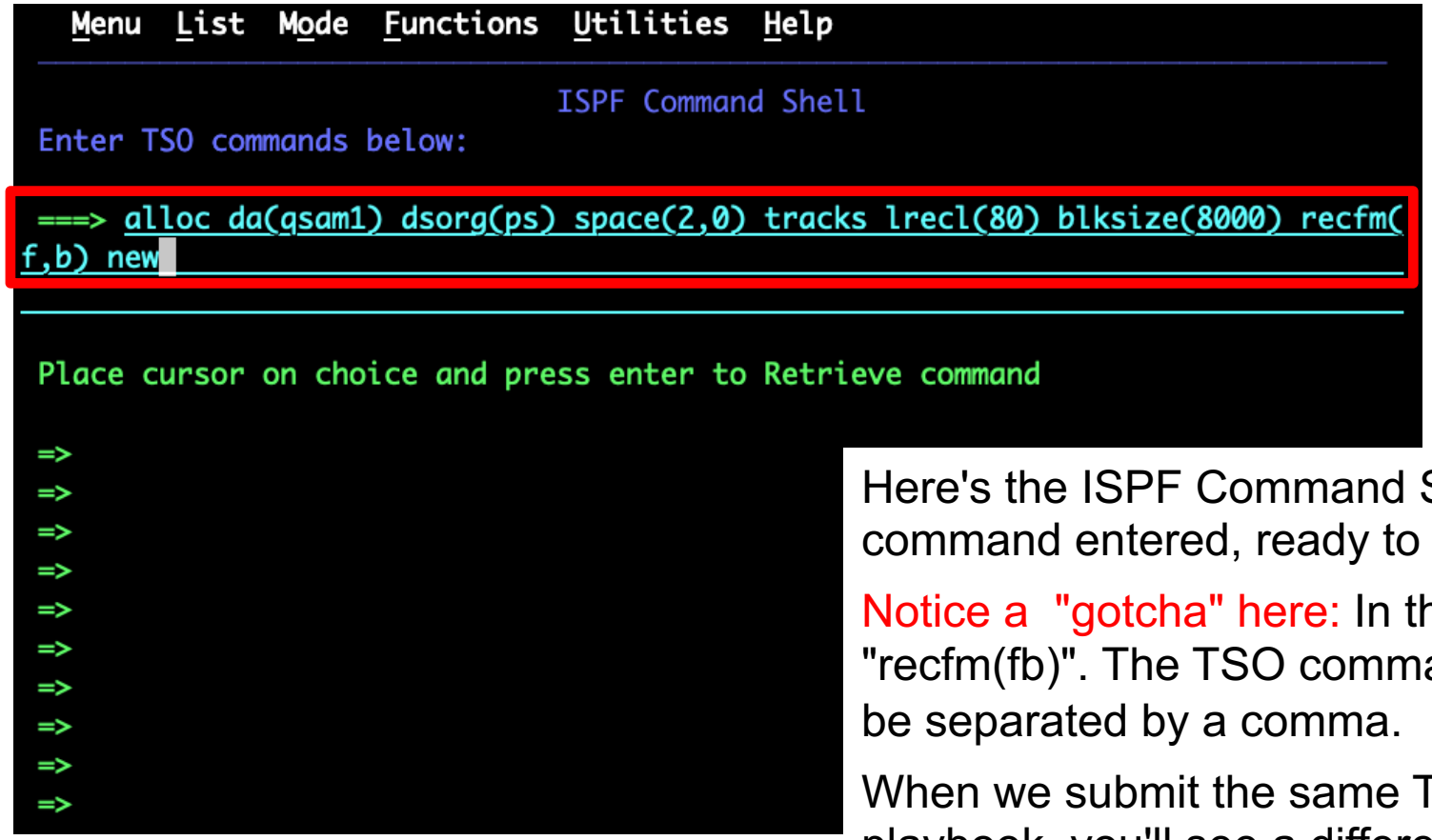
Just as Ansible is an abstraction over z/OS, ISPF is an abstraction over TSO.

You can create a z/OS data set with a TSO command, either from the bare TSO UI or from the ISPF Command Shell panel.

If you're accustomed to working on a *nix command line, you might find this easier than filling in fields on an ISPF panel. Either way, it's good to understand how to do it.

© 2023 by Innovation In Software Corporation

USING TSO TO CREATE A DATA SET



Here's the ISPF Command Shell panel with a TSO ALLOCATE command entered, ready to execute.

Notice a "gotcha" here: In this context, you can't type "recfm(fb)". The TSO command parser requires the "f" and "b" to be separated by a comma.

When we submit the same TSO command from an Ansible playbook, you'll see a different "gotcha".

You're in the Land of Gotchas.

© 2026 by Innovation In Software Corporation

USING TSO TO CREATE A DATA SET

```
                          Data Set Information
Command ===>

Data Set Name . . . . : IBMUSER.QSAM1

General Data                          Current Allocation
 Management class . . : **None**       Allocated tracks  . : 1
 Storage class  . . . : SCBASE         Allocated extents . : 1
  Volume serial . . . : USRVS1
  Device type . . . . : 3390
 Data class . . . . . : **None**
  Organization  . . . : PS             Current Utilizatior
  Record format . . . : FB              Used tracks . . . . : 0
  Record length . . . : 80              Used extents  . . . : 0
  Block size  . . . . : 8000
  1st extent tracks . : 1
  Secondary tracks  . : 1             Dates
  Data set name type  :                Creation date . . . : 2026/01/25
  Data set encryption : NO             Referenced date . . : ***None***
                                       Expiration date . . : ***None***

  SMS Compressible  . : NO
```

After running the command, we see the data set was created with the attributes we specified.

© 2023 by Innovation In Software Corporation

LAB 9: USING TSO TO CREATE A QSAM DATA SET

Follow the instructions for Lab 9 in labs/labs.md in the course repo.

© 2023 by Innovation In Software Corporation

USING ZOS_TSO_COMMAND TO CREATE A DATA SET

```
---
# This playbook uses zos_tso_command to create a QSAM data set
# on the z/OS instance.

- name: Submit a TSO command to create a QSAM data set
  hosts: zos
  gather_facts: false
  collections:
    - ibm.ibm_zos_core
  environment: "{{ environment_vars }}"

  tasks:

    - name: Create a data set using TSO ALLOC
      ibm.ibm_zos_core.zos_tso_command:
        commands:
          - ALLOC DA(QSAM2) NEW CATALOG SPACE(1 1) TRACKS DSORG(PS) RECFM(F) LRECL(80) BLKSIZE(800)
      register: result

    - name: Display result of data set creation
      debug:
        msg: "{{ result }}"
```

Module zos_tso_command, part of ibm_zos_core, submits TSO commands from Ansible playbooks.

Here we use it to create a QSAM data set.

Gotchas:

1. The TSO command must be all on one line.
2. RECFM is just F; the presence of BLKSIZE tells the system the data set is blocked.
3. The subparameters of SPACE are separated by a space; commas are not allowed in this context.

© 2023 by Innovation In Software Corporation

USING ZOS_TSO_COMMAND

```
---
# This playbook uses zos_tso_command to obtain the user's TSO
# profile and list the datasets catalogued under that userid.

- name: Get TSO profile and listcat
  hosts: zos
  gather_facts: false
  collections:
    - ibm.ibm_zos_core
  environment: "{{ environment_vars }}"

  tasks:

    - name: Get TSO user profile and listcat
      ibm.ibm_zos_core.zos_tso_command:
        commands:
          - PROFILE
          - PROFILE PREFIX(IBMUSER)
          - LISTCAT LEVEL(IBMUSER)
        register: result

    - name: Display result
      debug:
        msg: "{{ result }}"
```

Module zos_tso_command isn't the first choice for creating data sets. For that, we'd normally use module zos_data_set.

Let's say you wanted to check the TSO profile of a user, and you wanted to get a list of all the data sets catalogued with that user's z/OS id as the high-level qualifier.

You don't have to log on to the z/OS instance to do that, as you can do it with the zos_tso_command module.

Here's a playbook that gets the default TSO profile, the specified user's TSO profile, and LISTCAT output for the specified user.

You need both the default profile and the user's profile because the user's profile will contain only overrides, and not all the settings. It inherits settings from the default profile. This command doesn't merge the two.

When you enter the command directly in TSO, it *does* merge the inherited defaults with the user profile settings.

© 2023 by Innovation In Software Corporation

USING ZOS_TSO_COMMAND

```
TASK [Get TSO user profile and listcat] ****************************************************************************************************************
changed: [mainframe]

TASK [Display result] ***************************************************************************************************************************************
ok: [mainframe] => {
    "msg": {
        "changed": true,
        "failed": false,
        "max_rc": 0,
        "output": [
            {
                "command": "PROFILE",
                "content": [
                    "IKJ56688I CHAR(0)  LINE(0)    PROMPT   INTERCOM   NOPAUSE MSGID   NOMODE  NOWTPMSG NORECOVER PREFIX(IBMUSER)  PLANGUAGE(ENU) SLANGUA",
                    "GE(ENU) VARSTORAGE(LOW)  ",
                    ""
                ],
                "failed": false,
                "lines": 3,
                "rc": 0,
                "stderr": ""
            },
            {
                "command": "PROFILE PREFIX(IBMUSER)",
                "content": [
                    ""
                ],
                "failed": false,
                "lines": 1,
                "rc": 0,
                "stderr": ""
            },
```

Here's part of the result returned from running playbook get_user_profile_and_listcat.

It shows the default profile settings and overrides for user IBMUSER, of which there are none at this time.

© 2023 by Innovation In Software Corporation

USING ZOS_TSO_COMMAND

```
{
    "command": "LISTCAT LEVEL(IBMUSER)",
    "content": [
        "NONVSAM ------- IBMUSER.BUILD.COBOL",
        "     IN-CAT --- CATALOG.VS01.TSO",
        "NONVSAM ------- IBMUSER.BUILD.OBJ",
        "     IN-CAT --- CATALOG.VS01.TSO",
        "NONVSAM ------- IBMUSER.COMMON.CACERT",
        "     IN-CAT --- CATALOG.VS01.TSO",
        "NONVSAM ------- IBMUSER.D142.T1019166.IMS15J11",
        "     IN-CAT --- CATALOG.VS01.TSO",
        "NONVSAM ------- IBMUSER.EX1.DATA",
        "     IN-CAT --- CATALOG.VS01.TSO",
        "NONVSAM ------- IBMUSER.HCD.MSGLOG",
```

Here's part of the result for the LISTCAT command. You get the data set name, the catalog in which it is catalogued, and the type of catalog entry. In this example, all the data sets shown are NONVSAM.

©2023 by Innovation In Software Corporation

LAB 10: USING ANSIBLE TO RUN A TSO COMMAND

Follow the instructions for Lab 10 in labs/labs.md in the course repo.

© 2023 by Innovation In Software Corporation

WHAT YOU KNOW AND WHAT YOU CAN DO

- You can create a QSAM dataset using a TSO command, ISPF panels, or an Ansible playbook.
- You can issue TSO commands from an Ansible playbook.

© 2023 by Innovation In Software Corporation

WHAT'S NEXT

- z/OS Subsystems and JES